

7. 함수

프로그래밍 언어에서 함수는 일련의 과정을 문(statement)으로 구현하고 코드 블록으로 감싸서 하나의 실행 단위로 정의한 것으로 자바 스크립트에서 함수 선언 방식은 선언식, 표현식에 의한 방식이 있으면 표현식은 화살표 함수로 표시 할 수 있습니다.

함수는 다음을 포함하고 있습니다,

- 매개변수(parameter) : 함수 내부로 입력을 전달받는 변수
 - 원시 타입은 변경이 불가능 하니, 파라미터에서 재할당을 통해 새로운 원시값으로 교체하여 기존 값의 변경이 되지 않습니다.
 - 객체 타입으로 인수를 받는다면, 참조 값이 복사되어 원본에도 영향을 줍니다.
- 인수 (argument) : 입력
- 출력 (return value) : 출력

```
function add(x, y) { // parameter
    return x+y; // return value
}

add(2,5); // argument

// 함수 리터럴 : 함수는 객체다
var f = function add(x, y) {
    return x+y;
}
```

- JavaScript는 함수 자체가 다른 함수의 인자가 될 수 있습니다. 즉 First Class Object
- 함수는 변수는 Scope를 결정 하고 private변수 또는 메소드 뿐만 아니라 함수의 특징을 이용 하여 public 속성과 메소드를 제공 하며 자바 스크립트 모듈을 작성 합니다.
- 함수는 하나의 기능을 제공해야 합니다 -> 단일책임원칙

{% hint style="info" %} First Class Object

- 변수에 저장 할 수 있어야 한다.
- 함수를 파라미터로 전달할 수 있어야 한다.
- 함수의 반환값으로 사용할 수 있어야 한다.
- 자료 주소에 저장할 수 있어야 한다. {% endhint %}

1. 함수 생성자

- new 키워드로 호출되는 함수가 생성자
- 생성자로 호출된 함수의 this 객체는 새로 생성된 객체를 가리키고, 새로 만든 객체의 prototype 에는 생성자의 prototype이 할당된다
- 전역범위는 전체를 의미 하며 지역 범위는 정의 된 함수 내에서만 참조 되는 것을 의미 함
- 생성자에 명시적인 return 구문이 없으면 this가 가리키는 객체를 반환한다.\

```
function Person(name) {
    this.name = name;
}

Person.prototype.getName = function() {
    console.log(this.name);
};

const fPerson = new Person("홍당무");
fPerson.getName(); // 홍당무
```

- 생성자에 명시적인 return 문이 있는 경우에는 반환하는 값이 객체인 경우에만 그 값을 반환한다.

```
function car() {
    return '현대';
}

console.log(car()); // 현대
console.log(new car()); // 새로운 객체를 반환 --> car {}
```

```
function person() { this.someValue = 2; return { name: '한국인', someValue: this.someValue }; }
let pObj = new person(); console.log(person()); // {name: '한국인', someValue: 2}
console.log(pObj); // {name: '한국인', someValue: 2}
```

```
var pFuncPerson = person(); var pObjPerson = new person(); var pObjPerson01 = new person();
```

```
console.log ( pObjPerson == pFuncPerson ? '같음' : '틀림' ); // '틀림'
console.log ( pObjPerson == pObjPerson01 ? '같음' : '틀림' ); // '틀림'
console.log ( pObjPerson === pFuncPerson ? '같음' : '틀림' ); // '틀림'
console.log ( pObjPerson === pObjPerson01 ? '같음' : '틀림' ); // '틀림'
```

```
console.log ( pObjPerson == pObjPerson ? '같음' : '틀림' ); // '같음'
console.log ( pObjPerson01 === pObjPerson01 ? '같음' : '틀림' ); // '같음'
```

- new 키워드가 없으면 그 함수는 객체를 반환하지 않는다.

- new 로 생성하지 않는 경우

```
function Fun() {
    this.hasEyePatch = true; // 전역 객체를 준비! new 로 생성 하지 않으면 전역
    변수 취급
    firstName = "길동"; // 전역 객체를 준비!
    var var1 = "var1" ; // 내부 만 가능
    console.log(var1 )/ // var1
}
```

```
// Fun() 함수는 생성 되지 않고 실행이 됨 var fFun = Fun(); console.log(fFun); // undefined
console.log(hasEyePatch); // true , new 로 생성 하지 않으면 전역 변수 취급 console.log(firstName); // 길동
console.log(fFun.firstName); // Cannot read property 'firstName' of undefined
console.log(fFun.var1); // Cannot read properties of undefined (reading 'var1') console.log(var1); //
Uncaught ReferenceError: var1 is not defined // Fun 함수 내부에서만 사용 가능
```

- new 로 객체 생성 하는 경우 \

```
function FunObj() {
  this.hasEyePatchObj = true; // 속성을 가지고 있음
  firstNameObj = "길동";      // 전역 객체를 준비! 속성이 아님
  var var1Obj = "var1";       // 내부 변수로 내부 만 사용 가능
  console.log(var1Obj) // var1
}

// 객체로 생성됨
var fFunObj = new FunObj();

// this로 명시한 것만 속성으로 가짐
console.log(fFunObj); // FunObj {hasEyePatchObj: true}
console.log(hasEyePatchObj);
// Uncaught ReferenceError: hasEyePatchObj is not defined
console.log(firstNameObj); // 길동
console.log(fFunObj.firstNameObj); // undefined
console.log(fFunObj.var1Obj); // undefined
console.log(var1Obj);
// VM2028:1 Uncaught ReferenceError: var1 is not defined
// Fun 함수 내부에서만 사용 가능
```

2. 함수 정의

- 함수 선언문
 - 로딩 시점에 VO(Variable Object)에 적재 하므로 어느 곳에서도 접근 가능
- 함수 표현식
 - Runtime시점에 적재
- Function 생성자 함수
- 화살표 함수 (ES6)

함수 선언식 (Function Declaration)	function fun() { ... }
기명 함수 표현식 (Named Function Expression)	var fun = function fun() { ... }
익명 함수 표현식 (Anonymous Function Expression) -> 함수 리터럴	var fun = function() { ... }
기명 즉시실행함수(named immediately-invoked function expression)	(function fun() { ... } ());
익명 즉시실행함수(immediately-invoked function expression)	(function() { ... })();

```
// 1. 함수 선언문
function add(x,y) {
    return x+y;
}

// 2. 함수 표현식
var add = function(x,y) {
    return x+y;
}

// 3. Function 생성자 함수
var add = new Function('x', 'y', 'return x+y');

// 화살표 함수 (ES6)
var add = (x,y) => x+y;
```

2-1. 함수 매개변수

나머지 매개변수는 전개 연산자(...)로 표현한 매개변수

```
// 1. 다른 매개변수와 사용 - 배열 형태
function fn00(a, b, ... args) {
    console.log(a, b, args); // > 1 2 [3, 4, 5]
    // ... ;
}
fn00(1, 2, 3, 4, 5);

// 2. 단독 사용
function fn00(... args) {
    console.log(args); // > [1, 2, 3, 4, 5]
}
fn00(1, 2, 3, 4, 5);

// 3. 매개변수의 기본값 - 기본값 사용안할 때
function fn00(a, b) {
    console.log("a = " + a);
    console.log("b = " + b); // undefined
};
add(1);

// 3. 매개변수의 기본값 - 기본값 사용할 때
function fn00(a, b=100) {
    console.log("a = " + a);
    console.log("b = " + b); // 1000
};
add(1);
```

3. 함수 선언식

함수 선언식은 표현식이 아닌 문(statement)입니다.

1. function 뒤에 함수명
2. 소괄호 () 안에 매개변수를 쉼표로 작성
3. 중괄호 {} 안에 수행되는 명령 작성
4. return 문을 작성하지 않으면 자동으로 undefined를 반환

자바스크립트 엔진은 생성된 함수를 호출하기 위해 함수 이름과 동일한 이름의 식별자를 암묵적으로 생성하고, 거기에 함수 객체를 할당합니다.

즉, 우리는 함수 이름으로 호출하는 것처럼 보였지만 사실은 함수 객체를 가리키는 식별자로 호출하는 것 입니다.

```
function 함수명 ( 매개변수1,
                 매개변수2,
                 ... ,
                 매개변수3 ) {
    ...
}
```

다음 문장을 작성 하세요.

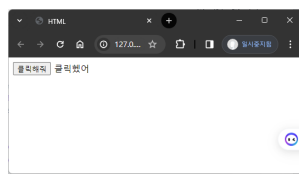
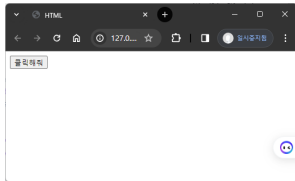
```
```html
<body>
 <button id="clickme"
 onclick="fnOnClick()">클릭해줘</button>
 <Label id="clickmelabel" ></Label>
</body>
<script>
 // 함수 선언식으로 함수 선언
 function fnOnClick() {
 /**
 * 자바 스크립트의 내장 객체인 document에 있는 querySelector() 메서드
 * 를 사용해서 clickme 버튼을 클릭 하면 clickmelabel에 표시
 */
 document.querySelector("#clickmelabel").innerHTML = "클릭했어";
 }
</script>
```
</script>
```

실행전

실행후

실행전

실행후



4. 함수 표현식

자바스크립트 함수는 객체 타입의 값입니다. 즉, 값처럼 변수에 할당하거나 프로퍼티가 되거나 배열이 될 수 있는 성질을 갖는 객체를 일급 객체라고 하는데 이런 성질을 사용하는 방식을 함수 표현식이라 합니다.

표현식은 함수명 없이 선언한 후 객체 변수에 저장하는 방법으로 변수명이 함수명이 되는 것입니다.

- 함수 리터럴의 함수 이름은 생략할 수 있고, 이를 익명 함수라 합니다.
- 함수 표현식의 함수 리터럴은 함수 이름을 생략하는 것이 일반적입니다.

```
// 함수 리터럴을 함수 표현식으로 이야기함
var 변수명 = function ( 매개변수1,
                        매개변수2,
                        ...,
                        매개변수3 ) {
    ...
}
```

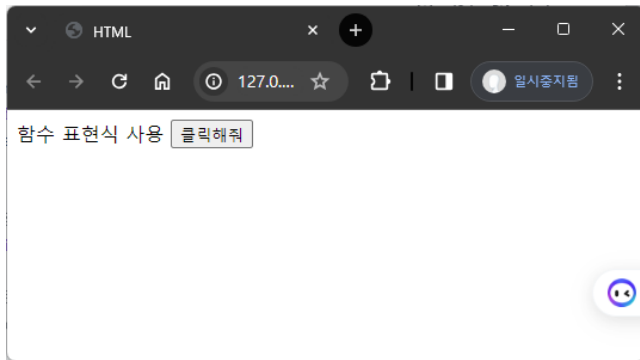
```
<body>
  함수 표현식 사용
  <button id="clickme"
    onclick="fnOnClickClickme()">클릭해줘</button>
  <Label id="clickmelabel" ></Label>
</body>
<script>
  // 함수 표현식으로 함수 선언
  const fnOnClickClickme = function fnOnClick() {
```

```

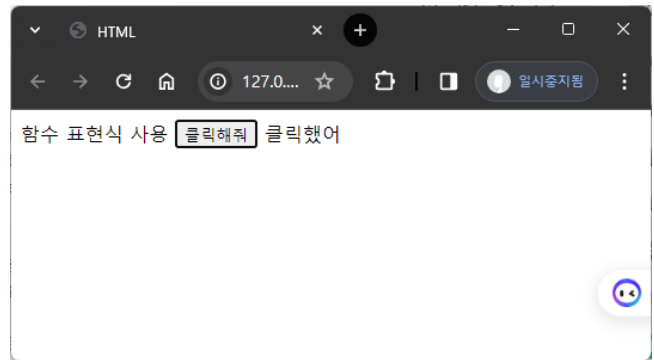
/**
 * 자바 스크립트의 내장 객체인 document에 있는 querySelector() 메서드
 * 를 사용해서 clickme 버튼을 클릭 하면 clickmelabel에 표시
 */
document.querySelector("#clickmelabel").innerHTML = "클릭했어";
}
</script>

```

실행전



실행후



5. 화살표 함수

함수 표현식은 ES6 (ECMAScript 2015)에 도입한 화살표 함수는 함수 선언시 function, return을 생략하고 화살표(=>)를 사용하여 간결히 표기합니다.

- Function 선언 대신 => 기호로 함수 선언
- 변수에 할당 하여 재 사용 할 수 있음
- 함수 내부에 return 문만 있는 경우 생략 가능생성자 함수로 사용할 수 없다.
- 기존 함수와 this 바인딩이 다르다.
- prototype 프로퍼티가 없다.
- arguments 객체를 생성하지 않는다.

```

변수명 = (매개변수1, ... 매개변수2 ) => { ... return }

```

```

// 전통적인 함수 선언
function fnAdd(pNum1, pNum2) {
  return pNum1 + pNum2;
}

```

```

// 함수 표현식 선언
const fnAdd = function fnAdd(pNum1, pNum2) {
  return pNum1 + pNum2;
}

```

```

// 화살표 함수
const fnAdd = (pNum1, pNum2) => {
  return pNum1 + pNum2;
}

```

```
// 화살표 함수 return 생략
const fnAdd = (pNum1, pNum2) => pNum1 + pNum2;

// 매개 변수 1개일 때 소괄호 생략
const fnAdd = pNum1 => pNum1 + 1;

// 매개 변수 없을 때
const fnAdd = () => {
  // 실행 명령
  // return 이 있으면 작성
};

// 단문 리턴 생략
const fnAdd = () => '단문리턴';

const fnOnClickClickme = () => {
  document.querySelector("#clickmelabel")
    .innerHTML = "클릭했어";
}
```

6. 즉시 실행 함수

함수의 정의와 동시에 즉시 실행되는 함수로 단 한번만 호출되며 다시 호출할 수 없습니다.

```
// 1. 즉시 실행 함수
(function() {
  console.log("즉시 실행 함수");
})();

// 2. 함수 표현식에 의한 명시적인 호출
var func = function() {
  console.log("함수 표현식에 의한 명시적인 호출");
};
func();

// 3. 변수에 즉시 실행 함수를 할당 후 실행
var add = (function() {
  var a = 1;
  var b = 2;
  return a+b;
})();
console.log(add); // 3

var app = (function() {
  var name = "홍길동";
  return {
    rName : name
  };
})();
```



```

console.log(app().rName); // 홍길동

// 4. 기명 함수 사용
var fPrintName= function(name) {
    console.log("나의 이름은 "+ name + "이다");
};

fPrintName("홍길동"); // 나의 이름은 홍길동이다

(function(name) {
    console.log("즉시 실행 :: 나의 이름은 "+ name + "이다");
})("홍길동"); // 즉시 실행 :: 나의 이름은 홍길동이다

```

7. 익명 함수이용 한 scope영역 관리

```

const fFun= {
    fFun01: function() {
        let name = "홍";
        return name;
    },
    fFun02: function(){
        let name = "당무";
        return name;
    },
    fFun03: function(){
        return this.fFun01() + this.fFun02();
    }
}

console.log(fFun.fFun01()); // 홍
console.log(fFun.fFun02()); // 당무
console.log(fFun.fFun03()); // 홍당무

```

8. parameters (ES6)

```

// 1. default parameters ( ES6 )
function fPrint(mag, actionType = '홈') {
    console.log(`메세지 : ${mag}, 출처:${actionType}`);
}

fPrint('안녕');
fPrint('안녕', '회사');

// 2. rest parameters ( ES6 ) : 파라미터를 펼침연산자로 넘김
function fPrint(...mag) {
    console.log(`메세지 : ${mag}`);
}

fPrint('안녕', '회사'); // 메세지 : 안녕,회사

```

9. 중첩 함수

함수 내부에 정의된 함수를 중첩 함수(Nested Function) 또는 내부 함수(Inner Function) 이라고 하며 외부 함수 내부에서만 호출 할 수 있습니다.

```
function outer() {  
  var x = 1;  
  
  // 중첩 함수  
  function inner() {  
    var y = 2;  
    // 외부 함수의 변수를 참조할 수 있다.  
    console.log(x+y); // 3  
  }  
  
  inner();  
};  
  
outer();
```

10. 콜백 함수

함수의 매개변수를 통해 다른 함수의 내부로 전달되는 함수를 콜백 함수라고 하며, 매개 변수를 통해 함수의 외부에서 콜백함수를 전달받는 함수를 고차 함수라고 한다.

```
function repeat(n, f) {  
  for(var i = 0; i < n; i++) {  
    f(i);  
  }  
};  
  
var logAll = function(i) {  
  console.log(i);  
};  
  
repeat(5, logAll); // 0 1 2 3 4
```

11. 고차 함수

다른 함수를 반환 하는 함수로 라메터 단일 책임의 원칙 적용 할 수 있습니다.

```
const car01 = {  
  name: '소나타',  
  compony: '현대자동차'  
};  
  
const car02 = {
```

```

    name: '그랜저',
    compony: '현대자동차'
  };

  const address = {
    post: '130-111',
    address: '서울시 용산구',
    detailAddress: '영업부'
  };

  function setCarAddress(pAddress, pCar) {
    let { post, address, detailAddress } = pAddress;
    let rnObj = {
      post,
      address: address + detailAddress
    };
    return {...rnObj, ...pCar};
  }

  console.log(setCarAddress(address, car01));
  // { post: '130-111',
  //   address: '서울시 용산구영업부',
  //   name: '소나타',
  //   compony: '현대자동차' }

  console.log(setCarAddress(address, car02));
  // { post: '130-111',
  //   address: '서울시 용산구영업부',
  //   name: '그랜저',
  //   compony: '현대자동차' }

```

11-1. 단일 책임의 원칙 적용

고차 함수를 이용해서 주소와 차이름을 분리 하여 단일 책임의 원칙 적용

```

function setCarAddress01(pAddress) {
  let { post, address, detailAddress } = pAddress;
  let rnObj = {
    post,
    address: address + detailAddress
  };
  // 고차 함수
  return (car) => { return {...rnObj, ...car} };
};

console.log(setCarAddress01(address)(car01));
// { post: '130-111',
//   address: '서울시 용산구영업부',
//   name: '소나타',
//   compony: '현대자동차' }

```

```
console.log(setCarAddress01(address)(car02));
// { post: '130-111',
//   address: '서울시 용산구영업부',
//   name: '그랜저',
//   compony: '현대자동차' }
```

11-2. 고차 함수를 이용 해서 변수 저장 하는 방법

```
var setAddress = setCarAddress01(address);

console.log(setAddress(car01));
console.log(setAddress(car02));
```

12. Curring

- 함수 하나가 n개의 파라미터 인자를 받는 것을 n개의 함수로 받는 방법
- 매개변수 일부를 적용하여 새로운 함수를 동적으로 생성하면 이 동적 생성된 함수는 반복적으로 사용되는 매개변수를 내부적으로 저장하여, 매번 인자를 전달하지 않아도 원본함수가 기대하는 기능을 채워 놓는다

```
// 1. 템플릿 리터럴 사용
const fprint = function ( a, b ) {
  return `${a} ${b}`;
};

console.log(fprint('이건', '뭐지 ....'));      // 이건 뭐지 ....

// 2. 고차 함수 사용
const fCurryPrint = function(a) {
  return function(b) {
    return `${a} ${b}`;
  };
};

console.log(fCurryPrint('이건')('뭐지 ....')); // 이건 뭐지 ....

// 화살표 함수로 표현
const fCurryPrintArray = a => b => `${a} ${b}`;
console.log(fCurryPrintArray('이건')('뭐지')); // 이건 뭐지 ....

// Curring
const fAdd = function ( a, b ) {
  return a+b;
};

const fAddCurry = function(f) {
  return function(a) {
    return function(b) {
```

```
        return f(a, b);
    };
};
const sum = fAddCurry(fAdd);
console.log(sum(1)(2));    // 3

// 동일한 기능을 함
fAddCurry(fAdd)(1)(2);    // 3
```